GWC Academy

PYTHON PROGRAMMING – BEGINNER TO ADVANCED

Live Instructor-Led | Practical & Project-Based | 60 Hours

Course Duration	60 Hours
Recommended Duration	10–12 Weeks
Mode	Live Online / Classroom
Level	Beginner to Intermediate / Advanced Foundation
Prerequisites	Basic computer knowledge; no prior programming experience required
Technologies	Python, SQLite, MySQL, Flask, FastAPI, Django
Additional Exposure	NumPy, Pandas, Matplotlib, Git & GitHub
Projects	2 Mini Projects + 1 Major Project

Course Overview

A structured Python programming course designed to build strong programming fundamentals and progress through object-oriented programming, data structures, file handling, databases, APIs, web frameworks, basic data analysis, testing and version control. The course emphasizes hands-on coding and project work.

Detailed Module-Wise Curriculum

Module 1: Introduction to Python & Development Environment

- Introduction to Programming
- Introduction to Python
- History and Features of Python
- Applications of Python
- Python Versions
- Installing Python
- Python Interpreter
- How Python Code is Executed
- Python IDEs
- VS Code / PyCharm Setup
- Running Python Programs
- Python Virtual Environment
- pip Package Manager
- Installing and Managing Python Packages
- Introduction to Jupyter Notebook

Module 2: Python Fundamentals & Syntax

- Python Syntax and Indentation
- Comments, Keywords and Identifiers
- Variables and Naming Conventions
- Multiple Variable Assignment
- Constants
- Input and Output
- print() and User Input
- Type Checking, Conversion and Casting
- Escape Sequences
- Numbers: Integer, Float and Complex
- Boolean and None Type
- Arithmetic, Assignment and Comparison Operators
- Logical, Identity and Membership Operators
- Bitwise Operators
- Ternary Operator

Module 3: Control Flow & Decision Making

- if, if-else and if-elif-else
- Nested Conditions
- Conditional Expressions
- match-case Basics
- for Loop
- while Loop
- Nested Loops
- Loop else
- break, continue and pass
- assert Statement
- Practical Logic-Building Exercises

Module 4: Python Data Structures

- Lists and Nested Lists
- Tuples and Tuple Operations
- Sets and Set Operations
- Dictionaries and Dictionary Methods
- Nested Dictionaries
- Stack Basics
- Queue Basics
- collections Module
- deque, Counter and defaultdict
- OrderedDict Overview

Module 5: Strings & String Manipulation

• String Creation, Indexing and Slicing
• String Immutability
• String Methods
• String Formatting and f-Strings
• Searching and Replacing Text
• Splitting and Joining Strings
• String Comparison
• Practical String Programs

Module 6: Functions & Functional Programming

• Defining and Calling Functions
• Parameters and Arguments
• Return Statement
• Positional and Keyword Arguments
• Default Arguments
• *args and **kwargs
• Local and Global Scope
• global Keyword
• Recursive Functions
• Lambda Functions
• map(), filter() and reduce()
• zip() and enumerate()

Module 7: Comprehensions, Iterators & Generators

• List, Dictionary and Set Comprehensions
• Nested and Conditional Comprehensions
• iter() and next()
• Iterator Protocol
• Generators
• yield Keyword
• Generator Functions and Expressions
• Memory-Efficient Programming

Module 8: Object-Oriented Programming in Python

• Classes and Objects
• Instance Variables and Methods
• Class Variables
• Class Methods
• Static Methods
• Constructors and __init__
• self Keyword
• Encapsulation
• Inheritance and Types of Inheritance
• Method Overriding
• Polymorphism
• Abstraction
• Property Decorators
• Special / Magic Methods
• __str__ and __repr__

Module 9: Modules, Packages & Standard Library

• Creating and Importing Modules
• from ... import and Aliases
• __name__ and __main__
• Creating Packages
• os and sys
• pathlib and shutil
• datetime and time

• random, math and statistics
• json and csv
• re Introduction
• subprocess Basics
• Command-Line Arguments

Module 10: Exception Handling & Logging

• Errors vs Exceptions
• try, except, else and finally
• Multiple Exceptions
• Raising Exceptions
• Custom and User-Defined Exceptions
• Exception Handling Best Practices
• Introduction to Logging
• Log Levels
• Basic Application Logging

Module 11: File Handling & Data Formats

• Opening and Closing Files
• File Modes
• Reading, Writing and Appending
• File Cursor and Positions
• Context Managers and with Statement
• Creating, Updating, Renaming and Deleting Files
• CSV Files
• JSON Files
• Reading and Writing JSON
• Working with ZIP Files

Module 12: Regular Expressions

• Regular Expression Basics
• Python re Module
• Patterns
• Character Classes
• Quantifiers
• Metacharacters
• Special Sequences
• search(), match(), findall()
• sub() and split()
• Practical Data Validation

Module 13: Database Programming with Python

• Database Concepts
• SQL Basics Required for Python
• SQLite
• Python with MySQL
• Database Connector Setup
• Creating Connections
• Executing SQL Queries
• CRUD Operations
• Parameterized Queries
• Commit and Rollback
• Database Exception Handling
• Basic Database Application Development

Module 14: API Development & API Consumption

• Introduction to APIs and HTTP
• GET, POST, PUT, PATCH and DELETE

• HTTP Status Codes
• JSON Request and Response
• requests Library
• Headers and Query Parameters
• Basic Authentication Concepts
• Flask Setup and Routing
• Request and Response
• Building Basic APIs
• FastAPI Setup and Routing
• Path and Query Parameters
• Request Methods
• Pydantic Models Basics
• Automatic API Documentation

Module 15: Django Web Development Fundamentals

• Django Architecture
• Installing Django
• Creating Projects and Apps
• Project Structure
• URLs and Routing
• Views
• Templates and Template Inheritance
• Models
• Django ORM Basics
• Database Migrations
• Django Admin
• Forms Basics
• User Authentication Basics
• Sessions
• Basic Database-Driven Web Application

Module 16: Multithreading & Date-Time Programming

• Process vs Thread
• threading Module
• Creating Threads
• Thread Lifecycle Basics
• Multiple Threads
• Synchronization Basics
• Thread-Safe Programming Concepts
• datetime Module
• date, time and datetime Objects
• Formatting and Parsing Dates
• timedelta

Module 17: Data Analysis Essentials

• Introduction to Data Analysis
• NumPy Arrays and Basic Operations
• Pandas Series and DataFrames
• Reading Data
• Basic Data Cleaning
• Basic Data Analysis
• Matplotlib
• Basic Charts and Visualizations

Module 18: Development Tools, Testing & Version Control

• Python Project Structure
• requirements.txt and Dependencies
• Virtual Environment Best Practices
• unittest Basics

• pytest Introduction
• Debugging
• Code Quality Basics
• Git Fundamentals
• GitHub Setup
• Repositories and Commits
• Basic Branching
• git add, commit and push
• Managing Python Projects on GitHub

Module 19: Projects & Practical Implementation

• Hands-on implementation throughout the course
• 2 Mini Projects
• 1 Major Project
• Project planning and requirement breakdown
• Applying Python fundamentals and OOP
• Database integration
• API integration or development
• Exception handling and modular structure
• Testing and debugging
• GitHub project management
• Project review and presentation

Practical Learning Structure

Hands-On Coding: Coding exercises and instructor-guided implementation throughout the course.

Assignments: Module-wise practical exercises covering Python, OOP, files, databases, APIs and web development.

Projects: 2 Mini Projects + 1 Major Project.

Interview Preparation: Coding practice and interview-oriented revision across Python, OOP, data structures, exceptions, databases and APIs.

Recommended Pricing

Plan	Fee	Purpose
Launch / Early Bird	■6,999	Recommended launch price
Regular Price	■8,999	Recommended standard website price
Premium Mentorship	■12,999	Optional future plan

Recommended Website Display: Regular Price ■8,999 with a limited Launch / Early Bird offer of ■6,999.

Course Outcome

Learners will build a strong Python foundation, work with OOP and data structures, handle files and exceptions, connect databases, consume and build basic APIs, gain introductory exposure to Django/Flask/FastAPI and data-analysis libraries, use Git/GitHub, and build portfolio-oriented projects.

GWC Academy

Learn. Build. Grow.